```
import pandas as pd # tabular
import numpy as np # numeric
import matplotlib.pyplot as plt # visualization
from sklearn.model_selection import train_test_split # its a module
from sklearn.linear_model import LinearRegression #for linear regression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score # for errors
print("the libraries are import")
```

```
the libraries are import
```

```
# there are create the dataset
# there are all elements numbers are same quntity
data ={
    "study_Hours":[ 1,2,3,4,5,6,7,8,9,10],
    "Attendance": [50,55,60,65,70,72,78,82,88,92],
    "marks":[35,40,45,52,58,65,70,78,85,90]
}
df = pd.DataFrame(data)
print(df)
```

```
   study_Hours  Attendance  marks
0            1          50     35
1            2          55     40
2            3          60     45
3            4          65     52
4            5          70     58
5            6          72     65
6            7          78     70
7            8          82     78
8            9          88     85
9           10          92     90
```

```
# step 2 inspect the dataset
print("first five rows")
print(df.head())

print("\nDataset shape:")
print(df.shape)

print("\ncolumn names:")
print(df.columns)

print("\nDataset information ")
df.info()

print("\nStatistical summary:")
print(df.describe())

print("Missing values")
print(df.isnull().sum())

print("\nDuplicate row:")
print(df.duplicated().sum())
```

```
first five rows
   study_Hours  Attendance  marks
0            1          50     35
1            2          55     40
2            3          60     45
3            4          65     52
4            5          70     58

Dataset shape:
(10, 3)

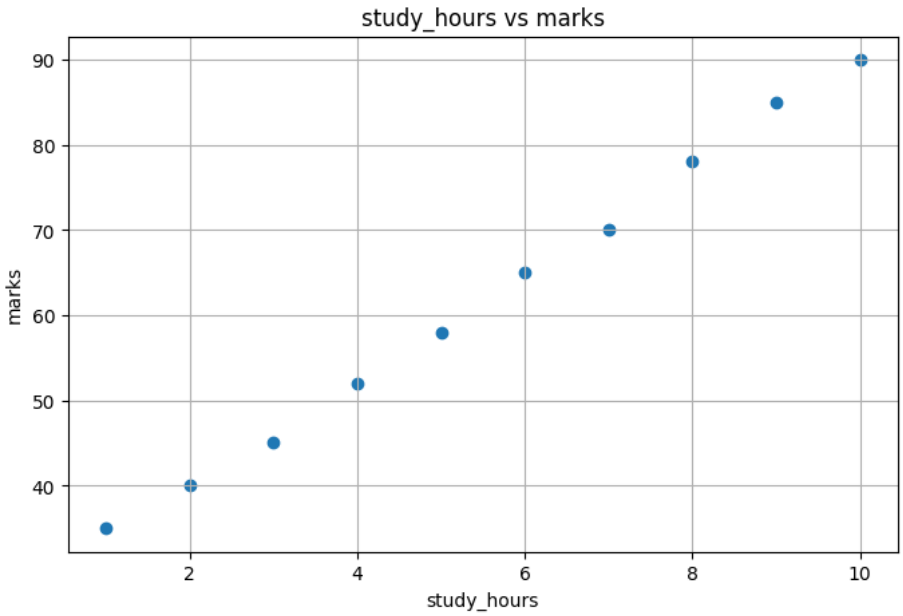
column names:
Index(['study_Hours', 'Attendance', 'marks'], dtype='object')

Dataset information
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 3 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   study_Hours  10 non-null     int64
 1   Attendance   10 non-null     int64
 2   marks        10 non-null     int64
dtypes: int64(3)
memory usage: 372.0 bytes

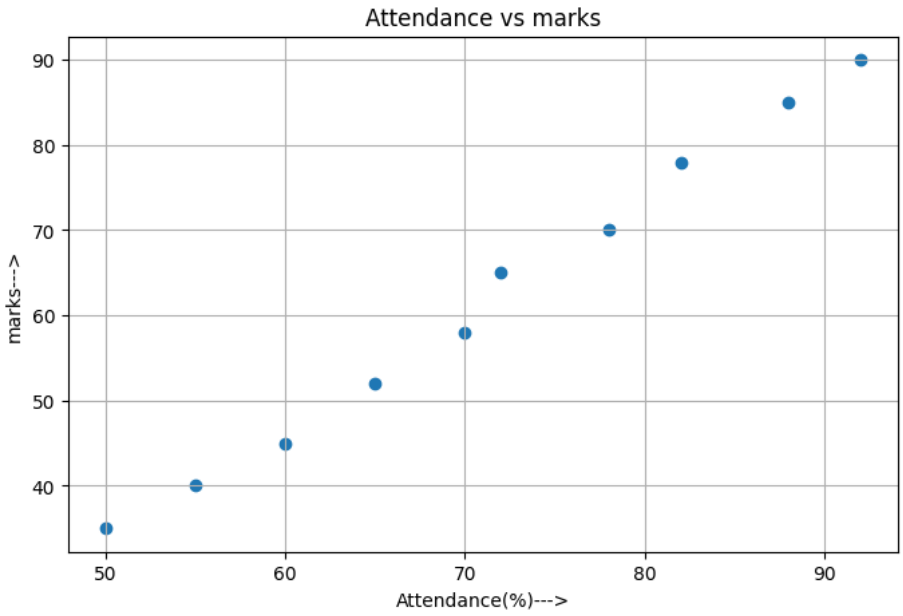
Statistical summary:
       study_Hours  Attendance      marks
count     10.00000   10.000000  10.000000
mean       5.50000   71.200000  61.800000
std        3.02765   13.966627  19.030969
min        1.00000   50.000000  35.000000
25%        3.25000   61.250000  46.750000
50%        5.50000   71.000000  61.500000
75%        7.75000   81.000000  76.000000
max       10.00000   92.000000  90.000000
Missing values
study_Hours    0
Attendance     0
marks          0
dtype: int64

Duplicate row:
0
```

```
plt.figure(figsize= (8,5))
plt.scatter(df["study_Hours"],df["marks"])
plt.xlabel("study_hours")
plt.ylabel("marks")
plt.title("study_hours vs marks")
plt.grid(True)
plt.show()
```




```
#attendance vs Marks
plt.figure(figsize=(8,5))
plt.scatter(df['Attendance'],df['marks'])
plt.xlabel("Attendance(%)--->")
plt.ylabel("marks--->")
plt.title("Attendance vs marks")
plt.grid(True)
plt.show()
```



```
#Saperate Features and Target
x=df[["study_Hours","Attendance"]]
y=df["marks"]

print("Features(X):")
print(x)
print("\nTarget(Y):")
print(y)
```

```
Features(X):
   study_Hours  Attendance
0            1          50
1            2          55
2            3          60
3            4          65
4            5          70
5            6          72
6            7          78
7            8          82
8            9          88
9           10          92

Target(Y):
0    35
1    40
2    45
3    52
4    58
5    65
6    70
7    78
8    85
9    90
Name: marks, dtype: int64
```

```
#split of train testing
x_train, x_test, y_train, y_test = train_test_split(
    x,
    y,
    test_size=0.2,
    random_state=42
    )

print("Training samples :", len(x_train))
print("Testing samples :", len(x_test))
```

```
Training samples : 8
Testing samples : 2
```

```
# building and creating the linear regression model
# step:1- create the model

model = LinearRegression()

# train the model
model.fit(x_train, y_train)
```

▾ LinearRegression

```
LinearRegression()
```

```
#Inspect Learned Parameters
print("cofficients:") # balance karne ke liye0
print(model.coef_)
print("\nIntercepts:")
print(model.intercept_)
```

```
cofficients:
[ 8.78650647 -0.56377079]

Intercepts:
53.39741219963015
```

```
#Prediction
#pridict the test data
y_pred=model.predict(x_test)

print("Actual Marks:")
print(y_test.values)

print("\nPredicted Marks:")
print(y_pred)
```

```
Actual Marks:
[85 40]

Predicted Marks:
[82.86414048 39.96303142]
```

```
camparison =pd.DataFrame({
    "Actual Marks":y_test.values,
    "Predicted Marks":y_pred
})
print(camparison)
```

```
   Actual Marks  Predicted Marks
0            85        82.864140
1            40        39.963031
```

```
# mean absolute error(MAE)
mae=mean_absolute_error(y_test,y_pred)
print("Mean Absolute Error:",mae)
```

```
Mean Absolute Error: 1.0864140480591509
```

```
#Mean Squared Error:
mse=mean_squared_error(y_test,y_pred)
print("Mean Squared Error:",mse)
```

```
Mean Squared Error: 2.281631281155925
```

```
r2=r2_score(y_test,y_pred)
print("R2 Score:",r2)
```

```
R2 Score: 0.9954930740125315
```

```
#After traning,the

new_student =pd.DataFrame({
    "study_Hours":[6],
    "Attendance":[90]
})
Predicted_Marks = model.predict(new_student)

print("study Hours:",new_student["study_Hours"].iloc[0])
print("Attendance:",new_student["Attendance"].iloc[0])
print("Predicated marks:",round(Predicted_Marks[0],2))
```

```
study Hours: 6
Attendance: 90
Predicated marks: 55.38
```

```
import joblib
import sklearn

# Save the train data
joblib.dump(model, "student_marks_model.pkl")

#Check the installed scikit-learn version
print("scikit-learn version:", sklearn.__version__)
print("Model saved successfully!")
```

```
scikit-learn version: 1.6.1
Model saved successfully!
```

```
from google.colab import files
files.download("student_marks_model.pkl")
```

Start coding or generate with AI.